2026 · PROJECT PORTFOLIO

안정재

LLM 애플리케이션 · 데이터 엔지니어

공공데이터 지도 서비스의 수집·검증·AI 파이프라인을 설계해 두 달간 매일 운영했고, 기능마다 정답이 있는 평가로 숫자를 남겼습니다. 채용공고 추출 파이프라인에서는 평가 장치 자체의 결함을 네 번 찾아 고쳤고, 그 과정을 전부 재현 가능하게 공개했습니다. 만드는 것보다 재는 장치를 먼저 세우는 개발을 지향합니다.

명지대학교 융합소프트웨어학부 데이터사이언스전공

kevin2483@naver.com · github.com/kevin2483 · kevin2483.github.io

CareerFit 라이브 데모

careerfit-kevin2483.streamlit.app

PROFILE

프로필 및 보유 역량

학력

명지대학교 융합소프트웨어학부 데이터사이언스전공
2021.03 ~ 2027.02 (재학)

경력

레디포스트 · 백엔드·데이터 (미래내일 일경험, 2026.09 ~ 2026.10)
재개발·재건축 공공데이터 지도 서비스의 파이프라인·API·AI 설계와 운영

자격

ADsP 데이터분석 준전문가 (제49회)

핵심 역량

- LLM 파이프라인 설계와 근거 검증기
- 정답셋 기반 성능 평가 · 블라인드 풀링 · 신뢰구간
- 평가 재현성(make reproduce · CI) 설계
- 에이전트·도구 호출·MCP 구성
- 공공데이터 수집·전처리·검증 자동화
- 분산 처리 파이프라인과 운영 자동화

기술 스택

언어 · 데이터
Python SQL Pandas NumPy PostgreSQL

LLM · 에이전트
Claude API 구조화 출력 Function Calling MCP 프롬프트 설계 평가셋 설계 부트스트랩 CI

수집 · 백엔드
FastAPI REST API Selenium BeautifulSoup 공공데이터포털 API Streamlit

인프라 · 운영
AWS EC2 Docker GitHub Actions OpenTofu Ansible cron Doppler

빅데이터 · 모델링
PySpark Hive HDFS GCP BigQuery scikit-learn XGBoost SHAP Matplotlib Looker Studio

ACT 01 · 레디포스트

흩어진 재개발·재건축 공공데이터를 지도 한 장에 모으고, AI가 자료만으로 답하는 서비스

2026.09 - 2026.10 · 미래내일 일경험 · 팀 3명 중 백엔드·데이터 전담

OVERVIEW

서울·경기·인천의 정비사업 자료는 기관마다 다른 형식의 엑셀, CSV, 게시판, 고시문으로 흩어져 있고 같은 사업장도 기관마다 다르게 적혀 있었습니다. 영업팀이 제안할 구역을 감이 아니라 근거로 찾게 하려면 먼저 자료를 한 단위로 묶고, 검증을 통과한 것만 내보내고, AI가 그 자료 밖의 말을 하지 못하게 막는 장치가 필요했습니다.

ROLE

수집·검증 파이프라인 · 자동 운영 · API 서버 · AI 질의응답과 평가 체계

7 / 7

PM이 확정한 KPI 7개 전부 달성 · 측정 2026.10.13 ~ 10.26
운영 지표 4개는 실행 기록으로, AI 지표 3개는 정답이 있는 평가셋으로 쟀습니다

01

검증을 통과한 데이터만 공개

검증 53항목 · 작업 스키마에서 공개 스키마로 한 번에 교체 · 새벽 자동 실행 14일 연속 성공 · 실패·침묵 알림

02

AI는 자료가 준 값만 말한다

도구 11개 호출 결과만으로 답하고 숫자는 검증기가 대조 · 근거 정확도 75% → 95% · 자료 밖 질문에 지어낸 답 0건

03

정답이 있는 평가를 먼저

자연어 검색 110문 100% · 고시문 날짜 추출 79% · 남이 쓴 50문으로 재측정

ACT 01 · 레디포스트

원본은 그대로, 검증 통과한 것만 공개

01 수집
기관 5곳 · 수집원 17개
엑셀·CSV·게시판·화면

02 지역별 적재
원문 컬럼은 그대로
표준 컬럼을 옆에 붙임

03 표준화·통합
같은 지번은 한 구역
중복은 지우지 않고 1:N

04 검증
행 수·채움률·급감·연결 무결성
53항목, 하나라도 실패면 중단

05 공개·감시
통과 때만 교체 · 변경 감지 요약
실패·침묵 알림 · 백업

새 표는 작업 스키마에 만들고, 검증을 전부 통과했을 때만 한 트랜잭션으로 공개 스키마와 바꿉니다. 그래서 파이프라인이 도는 동안에도 API는 직전 데이터를 그대로 주고, 실패한 실행의 데이터는 어디에도 보이지 않습니다. 매일 새벽 AWS에서 cron이 돌고, 실패하면 슬랙으로, 아예 안 돌면 아침 침묵 감시가 알립니다.

운영 화면: 시군구 버블, 목록, 조건 검색, 오른쪽 AI 패널

1,922
구역 · 사업장 레코드 1,949건을 지번 단위로 통합

131만
실거래 건 · 60개월 · 법정동 단위로 구역에 연결

14 / 14
새벽 자동 실행 성공 · 2026.10.13 ~ 10.26 · 매일 03:30

91.9%
지도 좌표 연결 · 행정동 표기가 원인임을 찾아 42곳 보강

ACT 01 · 레디포스트

AI는 자료가 준 값만 말한다

자연어를 구조화 출력으로 검색 조건에 바꾸고, 대화는 읽기 전용 도구 11개를 불러 그 결과만으로 답합니다. 답에 적힌 숫자와 날짜는 검증기가 도구 결과와 대조하고, 추천·전망 같은 금지 표현은 검사기가 잡아 다시 쓰게 합니다. 표는 화면이 그리고 모델은 글로 옮기지 않습니다. 해석·브리핑은 밤에 미리 만들어 검증을 통과한 문장만 두고, 못 넘으면 규칙 문장으로 떨어집니다.

평가 항목	결과	비고
자연어 검색 · 110문	**100%**	1차 94.1% → 프롬프트·정규형 수정
대화 20문 · 맞는 도구 선택	**100%**	Opus 5.5 / Haiku 5.5 비교 뒤 모델 확정
답의 숫자가 전부 자료에 있음	**75% → 95%**	검증기가 잡은 실패 패턴을 규칙으로 되먹임
투자 질문 20문 · 주의 문구	**100%**	금지 표현 0 · 거절 대신 사실로 안내
자료 밖 질문 24문 · 지어낸 답	**0건**	자동 채점 8.3%, 사람이 읽으면 0%
남이 쓴 50문 · 미확인 숫자·금지 표현	**0건**	공개 Q&A 37 + 담당자 13 · 사람 채점 진행

검색 p95 4.9초 · 대화 p95 10.5초 · 건당 약 6~60원 · 일일 토큰 상한과 jsonl 로그

ACT 01 · 레디포스트

비정형 문서에서 날짜를 뽑고, 정답과 대조하다

고시문 날짜 추출 파일럿

인천과 서울 일부 사업장은 단계 날짜 원천이 없어 타임라인이 비어 있습니다. 토지이음 고시목록 47만 행에서 경기 사업장 50곳의 고시 81건을 찾아 본문을 넣고, 단계·날짜·고시번호를 구조화 출력으로 뽑았습니다. 뽑은 날짜마다 본문 구절을 그대로 인용하게 하고 인용이 본문에 없으면 버립니다. 정답은 경기도 공표자료의 날짜 142개 항목입니다.

	후보 없음	정확	7일 안	불일치	정확률
기준선 (고시일 그대로)	77	51	8	6	78.5%
Claude 추출 (교차 인용 포함)	70	57	8	7	**79.2%**

추출의 이득은 교차 인용입니다. 뒤 단계 고시 본문이 앞 단계 고시를 인용해 14개 중 12개를 맞혔습니다. 인용 검증에 걸려 버린 사건 0/151 · 81건 약 0.6달러. 7일 안 8개는 인가일이 첨부 PDF에만 있어 HTML로는 못 고칩니다. 후보를 못 찾은 70개가 더 큰 한계입니다.

기록한 사고 두 건

숨기지 않고 적는 것도 검증의 일부입니다. 두 건 모두 테스트·검증 항목·절차로 되먹였습니다.

경기 자료 ID 중복으로 이력이 섞임

분기 파일의 연번이 시군마다 1부터 다시 시작해 촉진구역 40곳에 다른 사업장 이력이 붙어 이틀 동안 노출. 이력 합계 테스트가 잡아 ID를 파일 행 번호로 바꾸고 중복 검증 항목을 추가.

배포 뒤 API 재시작 누락

원천을 바꾼 뒤 서버 프로세스를 재시작하지 않아 20시간 동안 서울 이력·개요가 빈 채로 노출. 틀린 값은 아니었지만 운영 절차에 재시작 단계를 넣고 KPI 기록에 남김.

다시 한다면. 두 사고 모두 사람이 기억해야 하는 절차에서 났습니다. 재시작을 배포 스크립트에 넣고, ID 규칙 검증을 수집 단계로 앞당기고, 고시문은 첨부 PDF까지 읽는 경로를 먼저 만들겠습니다.

Python · pandas · PostgreSQL · FastAPI · Claude API · MCP · AWS EC2 · Docker · GitHub Actions · OpenTofu · Ansible

ACT 02 · CareerFit

채용공고 역량 추출 — 정규식 베이스라인과 Claude 3종을 같은 정답셋으로 재다

2026.07 - 2026.10 · 개인 프로젝트 · 전체 설계·구현 · 정답셋 판정 전부 직접

OVERVIEW

채용공고 1,190건에서 요구 역량을 뽑는 일은 LLM에 맡기면 결과가 바로 나옵니다. 문제는 그 결과가 맞는지 확인할 방법이 없다는 점입니다. 그래서 추출기보다 **재는 장치**를 먼저 만들었습니다. 사람이 판정한 정답셋, 정규식 베이스라인, 공고 단위 부트스트랩 신뢰구간, 그리고 그 장치 자체를 의심하는 절차입니다. 평가 장치의 결함을 네 번 찾아 고쳤고, 두 번은 결론이 바뀌었습니다.

ROLE

수집 · 사전 설계(106 canonical / 288 alias) · 추출 · 정답셋 구축 · 평가 · 라우팅 시뮬레이션 · 에이전트 팀 · 데모 배포

github.com/kevin2483/Careerfit · careerfit-kevin2483.streamlit.app

4회

평가 장치 자체의 결함을 찾아 고친 횟수 · 정답셋 38건 · 네 방식 비교
최종 F1 opus 0.863 · sonnet 0.854 · 정규식 0.853 · haiku 0.828 (95% 신뢰구간 겹침)

01

블라인드 풀링으로 정답셋 재구축

네 시스템 후보를 출처 가리고 섞어 사람이 판정 · 후보 439건 중 127건 제외 · 정규식-LLM 격차 0.091 → 0.024, 2차 교정에서 opus 최하위 → 1위

02

"차이가 있다"를 "확정할 수 없다"로

부트스트랩 10,000회 · opus–정규식 +0.010 [–0.016, +0.037] · 확정되는 것은 haiku가 opus·sonnet보다 낮다는 것뿐

03

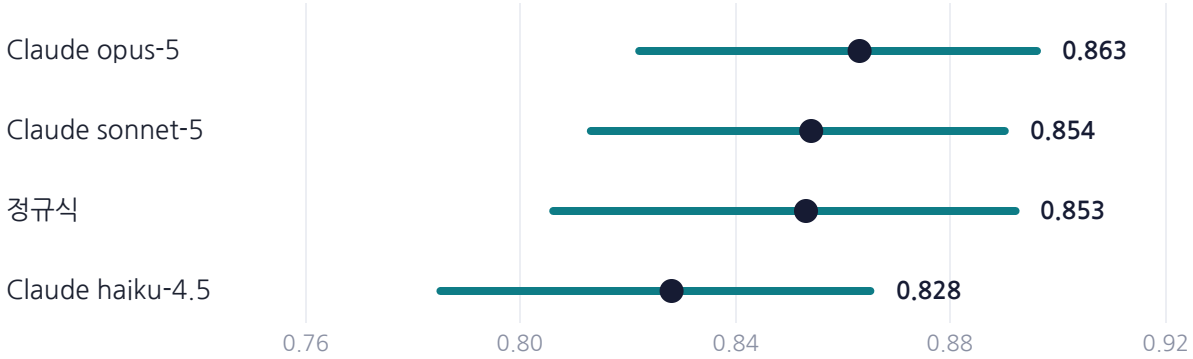
독립 리뷰가 찾은 네 번째 결함

sonnet 응답 1건이 출력 한도에서 잘려 빈 예측으로 채점 → 재호출 후 파레토 최적 합류 · 사전 등록한 프롬프트 변형 3종은 전부 실패, 그대로 공개

ACT 02 · CareerFit

재는 장치가 있으면 결론이 바뀌는 것도 보인다

네 방식의 F1과 95% 신뢰구간 · 정답셋 38건



짝지은 차이	점추정	95% CI	판정
opus – 정규식	+0.010	[–0.016, +0.037]	확정 불가
opus – haiku	+0.035	[+0.013, +0.059]	**확정**
sonnet – haiku	+0.027	[+0.001, +0.056]	**확정**

네 번의 교정

1	정답셋이 정규식 출력을 사람이 확인하는 방식이라 자기 채점 → 블라인드 풀링 재라벨
2	풀링 후보에 sonnet·opus가 빠져 단독 태그가 오탐 → 30건 추가 판정, opus 최하위 → 1위
3	사전 alias 충돌 4건·표기 오류 3건 → 사전 v2, 신뢰구간 도입, "우위" → "확정 불가"
4	sonnet 1건이 max_tokens에서 잘려 FN의 31% → 재호출, sonnet 0.845 → 0.854

+1.1σ

긴 공고 40%만 opus로 보내는 라우팅은 opus 수준(F1 0.863)을 비용 40%로 얻지만, 무작위 라우팅 대비 +1.1σ에 그쳐 "작동한다"고 쓰지 않았습니다

3 / 3

사전 등록한 haiku 프롬프트 변형(표현 예 · 라벨링 기준 · 구조화 출력) 전부 하락. 구조화 출력은 환각 0이지만 목록 안 오탐이 78 → 97

명시 기술은 네 방식 모두 F1 0.92~0.95로 동률이고 차이는 서술형 역량(opus 0.766 vs 정규식 0.736)에서 납니다. 한국어 조사가 정규식 단어 경계를 깨는 문제를 찾아 고쳐 추출량 +7.2%. 평가표 전부가 **make reproduce** 한 줄로 바이트 단위 재현되고 GitHub Actions가 매 push 검증합니다 (테스트 84건).

다시 한다면. 정답셋을 먼저 80건 이상으로 키우고, 예측 파일의 실패 행 검사를 첫날부터 채점기에 넣고, 프롬프트 실험은 정답셋을 개발용·평가용으로 나눈 뒤에만 하겠습니다.

ACT 03 · 띵커넥트

졸업선배 실시간 추천 매칭 서비스

2026.03 - 2026.06 · 캡스톤 프로젝트(팀) · 데이터 파트 전담 · 교내 SW 프로그래밍 대회 장려상 · github.com/kevin2483/DdingConnect

PROBLEM

가입 초기에는 사용자 이력 데이터가 거의 없어 추천을 시작할 근거가 부족한 콜드 스타트 상황이었습니다.

SOLUTION

협업 필터링 대신 내용 기반 가중치 매칭 모델을 직접 설계했습니다. 원티드 API 크롤링부터 DB 적재, FastAPI 서버 구축까지 데이터 파트 전체를 맡았고, 프론트·기획 팀원과는 API 명세를 먼저 합의한 뒤 매주 매칭 결과를 함께 보며 가중치를 조정했습니다.

Python · FastAPI · SQLAlchemy · BeautifulSoup · REST API · Jaccard

40 · 40 · 20

직무 · 기술 스택 · 목표 기업 가중치. 팀원과 매주 매칭 결과를 보며 항목별 실험으로 조정했고, 교수 피드백을 반영해 그 근거를 문서화했습니다

가중치 설계

관심 직무 유사도	직무 일치 100 / 개발군 내 50	**40%**
기술 스택 자카드	(교집합 / 합집합) × 100	**40%**
목표 기업 유사도	완전 일치 100점 · 팀원과 반복 실험으로 조합 조정	**20%**

ACT 04 · 데이터 파이프라인 두 건

수집부터 분석까지 직접 잇고, 결론은 데이터가 허락하는 만큼만

패션 커머스 데이터 플랫폼 · 수집 → BigQuery → MCP 질의 · 고객 세분화

2026.05 - 2026.07 · 직무 부트캠프(팀) · github.com/kevin2483/fashion-insight-platform

Selenium으로 무신사 랭킹을 수집하고 공휴일 API와 함께 BigQuery에 적재한 뒤 두 경로로 썼습니다. 하나는 Claude Desktop에서 자연어로 BigQuery를 질의하는 **MCP 서버**(JSON-RPC 직접 구현, SELECT 전용 가드). 다른 하나는 GA 공개 고객 데이터 9,996명의 RFM 4등급 세분화와 RF·XGBoost 예측입니다. 보정 전 정확도 98.5%가 우수고객 재현율 6%의 착시임을 확인하고 클래스 가중치 보정으로 94%까지 올렸습니다. 등급 라벨을 RFM 피처로 다시 예측하는 순환 구조는 한계로 적었습니다.

등급	고객	비중	평균 구매
VIP	1,745	17.5%	$444
우수고객	3,818	38.2%	$133
일반고객	3,612	36.1%	$66
이탈위험	821	8.2%	$22

우수고객 38.2%의 VIP 전환이 매출 증대의 가장 큰 지렛대입니다.

패션 수요 · 전환율 분산 분석 파이프라인

2026.06 · 빅데이터프로그래밍(팀) · GCP에 HDP 샌드박스 구성 · github.com/kevin2483/BigData_Final_Project

Crontab으로 Kaggle 로그와 기상청 ASOS API를 자동 수집하고, PySpark로 결합해 Parquet으로 HDFS에 적재한 뒤 Hive로 4종 집계, Matplotlib로 구간별 패턴을 확인했습니다. 수백만 건을 청크 스트리밍으로 처리해 메모리를 아꼈습니다.

비 오는 날 카테고리별 구매	Shoes 3,939 > Keds 2,481 · 악천후 유인 효과
기온대별 가격 민감도	Mild(10~20℃) 구매 3.3배 (Warm 대비)
10월 일별 매출 추이	10/18~24 최대 $9만 · 프로모션 기간과 일치
기온 구간별 전환율	약 0.5%로 일정 · 기온과 무관

날씨는 트래픽을 유인하는 보조 요인으로 보이고, 매출 피크는 프로모션 기간과 겹쳐 **날씨 단독 효과는 분리하지 못했습니다**. 그 한계까지가 결론입니다.

THANK YOU

만드는 것보다, 재는 장치를 먼저 세우겠습니다.

안정재
kevin2483@naver.com · github.com/kevin2483 · kevin2483.github.io

CareerFit 라이브 데모
careerfit-kevin2483.streamlit.app